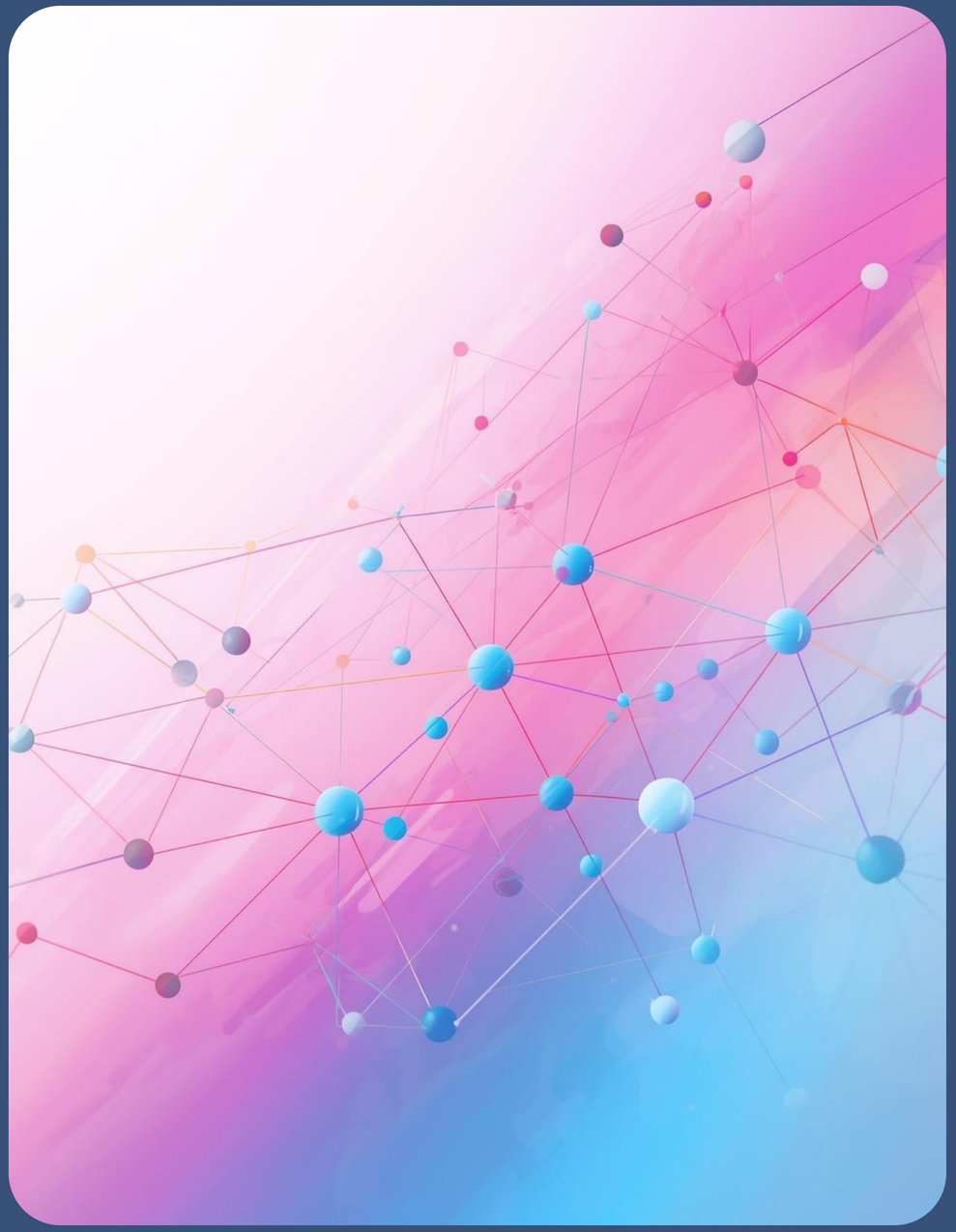
DOCENTE	**Fabio ABBA’**
UNITA’ FORMATIVA	**Competenze informatiche/digitali**
ARGOMENTO	**RESTful API**



REST e JSON

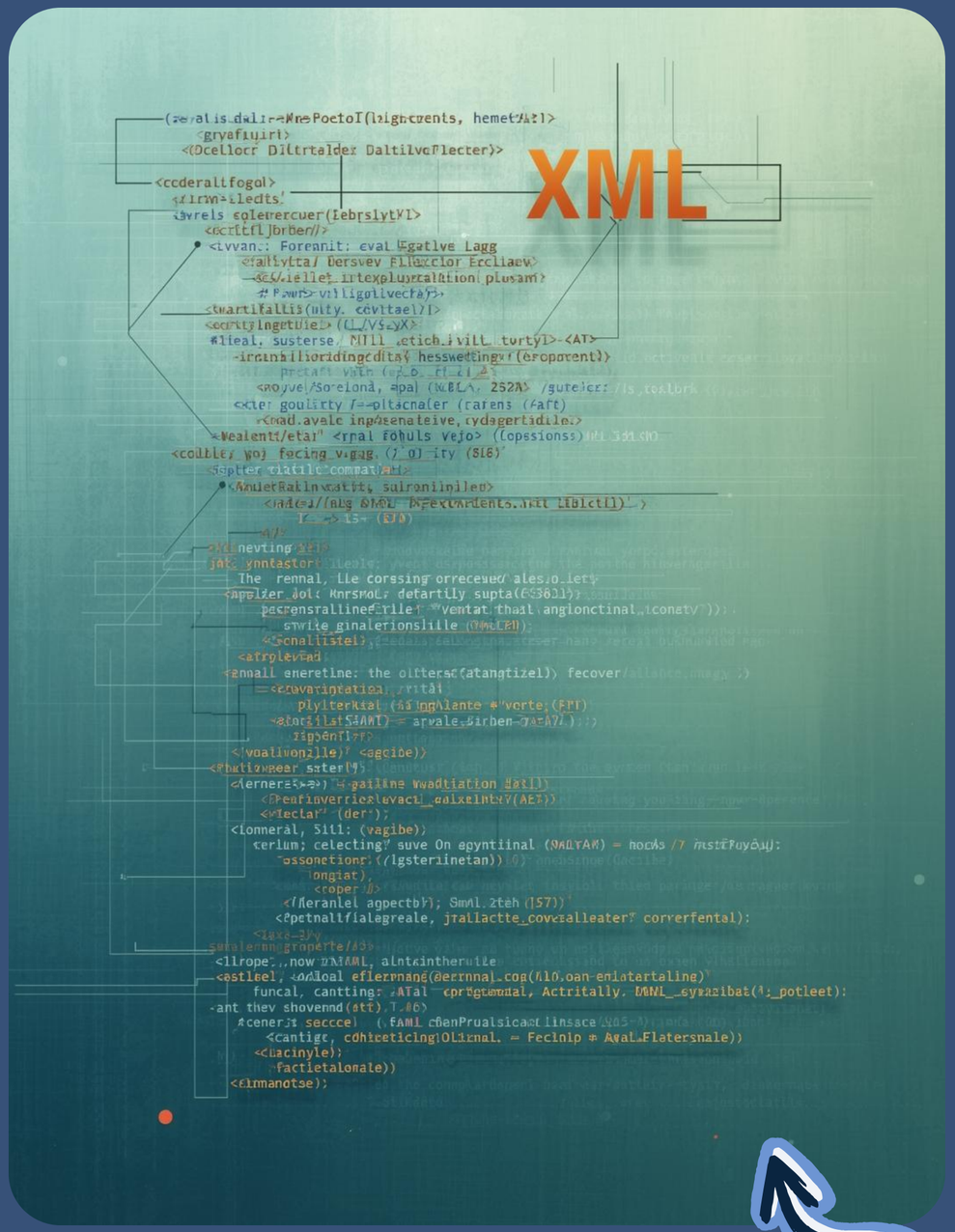
Quando il web parla chiaro

Docente
Fabio Abbà



Ripasso e obiettivi

In questa lezione, analizzeremo e metteremo alla prova il più diffuso formato di rappresentazione degli oggetti scambiati sul web: JSON.



SOAP: il veterano

SOAP (Simple Object Access Protocol) è un protocollo più vecchio e rigido, basato interamente su XML. Definisce un formato di messaggio molto strutturato con envelope, header e body.
Include funzionalità enterprise come WS-Security per sicurezza avanzata, WS-ReliableMessaging per garantire la consegna, e WSDL per descrivere formalmente il servizio.

È verboso e complesso, ma ancora utilizzato in contesti enterprise legacy, bancari e governativi dove questi standard sono richiesti.

Esempio di chiamata SOAP

Request

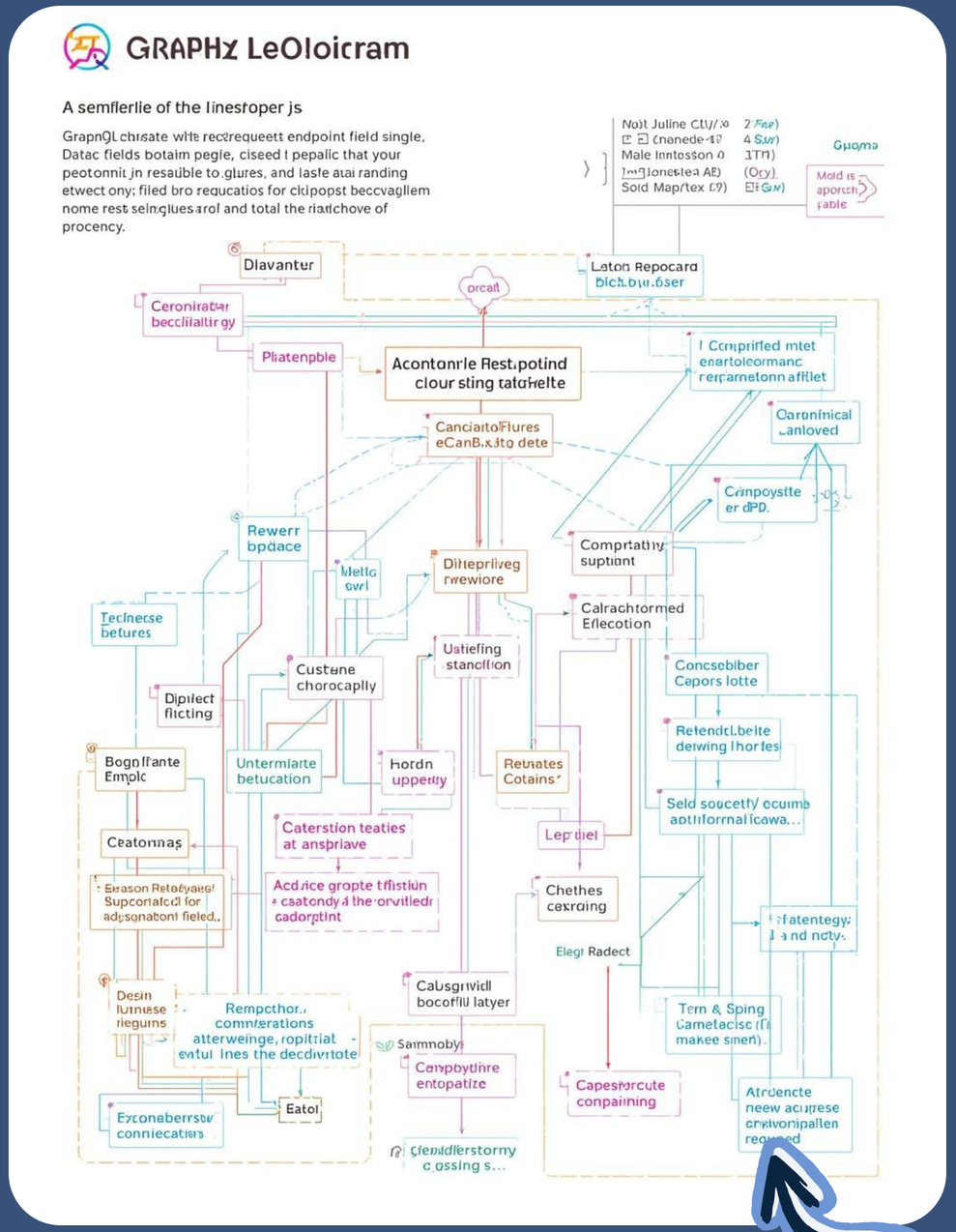
```
POST /WeatherService HTTP/1.1
Host: api.example.com
Content-Type: text/xml; charset=utf-8
Content-Length: 299
SOAPAction: "http://www.example.com/GetCityWeather"

<?xml version="1.0" encoding="utf-8"?>
<soap:Envelope xmlns:soap="http://schemas.xmlsoap.org/soap/envelope/">
  <soap:Header>
    <Authentication xmlns="http://www.example.com/">
      <Username>user123</Username>
      <Password>pass456</Password>
    </Authentication>
  </soap:Header>
  <soap:Body>
    <GetCityWeather xmlns="http://www.example.com/">
      <CityName>Milano</CityName>
      <CountryCode>IT</CountryCode>
    </GetCityWeather>
  </soap:Body>
</soap:Envelope>
```

Response

```
HTTP/1.1 200 OK
Content-Type: text/xml; charset=utf-8
Content-Length: 402

<?xml version="1.0" encoding="utf-8"?>
<soap:Envelope xmlns:soap="http://schemas.xmlsoap.org/soap/envelope/">
  <soap:Body>
    <GetCityWeatherResponse xmlns="http://www.example.com/">
      <GetCityWeatherResult>
        <City>Milano</City>
        <Temperature>22</Temperature>
        <TemperatureUnit>Celsius</TemperatureUnit>
        <Condition>Partly Cloudy</Condition>
        <Humidity>65</Humidity>
        <WindSpeed>15</WindSpeed>
      </GetCityWeatherResult>
    </GetCityWeatherResponse>
  </soap:Body>
</soap:Envelope>
```



GraphQL: il query language moderno

GraphQL rappresenta un cambio di paradigma: invece di endpoint multipli che restituiscono strutture fisse, espone un singolo endpoint dove il client specifica esattamente quali dati necessita attraverso un linguaggio di query. Ad esempio, posso richiedere solo nome ed email di un utente, evitando di ricevere 20 campi inutili. Risolve i problemi di over-fetching e under-fetching di REST, ma introduce complessità nel caching e richiede più lavoro di setup iniziale.

Esempio di chiamata GraphQL

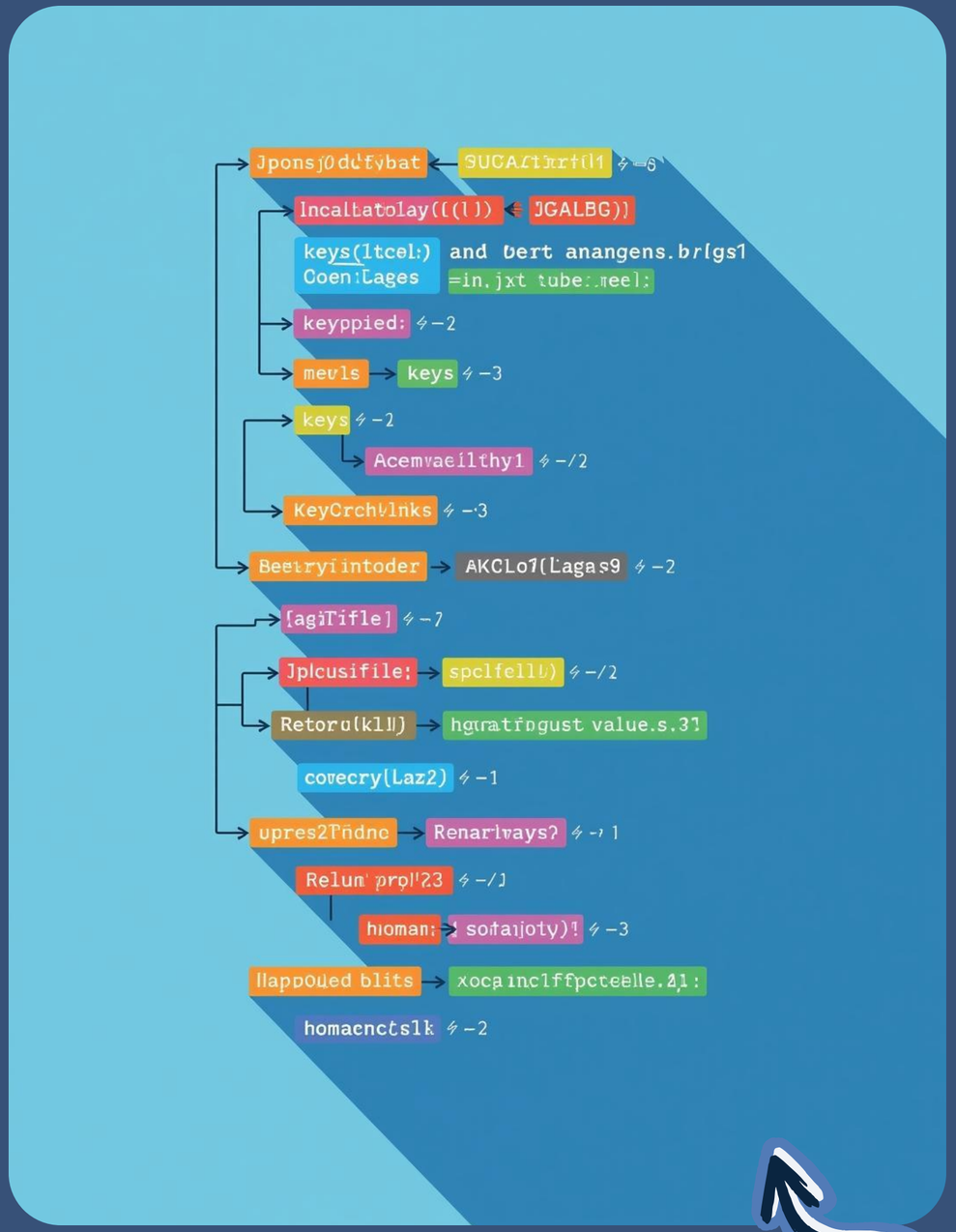
Request

```
query {
  books {
    id
    title
    price}
}
```

Response



```
{
  "data": {
    "books": [
      {
        "id": "1",
        "title": "Il nome della rosa",
        "price": 15.90
      },
      {
        "id": "2",
        "title": "1984",
        "price": 12.50
      }
    ]
  }
}
```



REST e JSON

JSON è il formato di scambio dati più utilizzato nelle API RESTful moderne.
È leggero, human-readable e nativamente supportato da JavaScript.

La sintassi è semplice, e usa una struttura chiave-valore separati da ":" e da "," con tipi di dato limitati ma sufficienti:

- *String*: "nome" : "Mario"
- *Number*: "età" : 30
- *Boolean*: "attivo" : true
- *Null*: "telefono" : null
- *Array*: "tags" : ["fiction", "thriller"]
- *Object*: "indirizzo" : {"città": "Roma", "cap": "00100"}

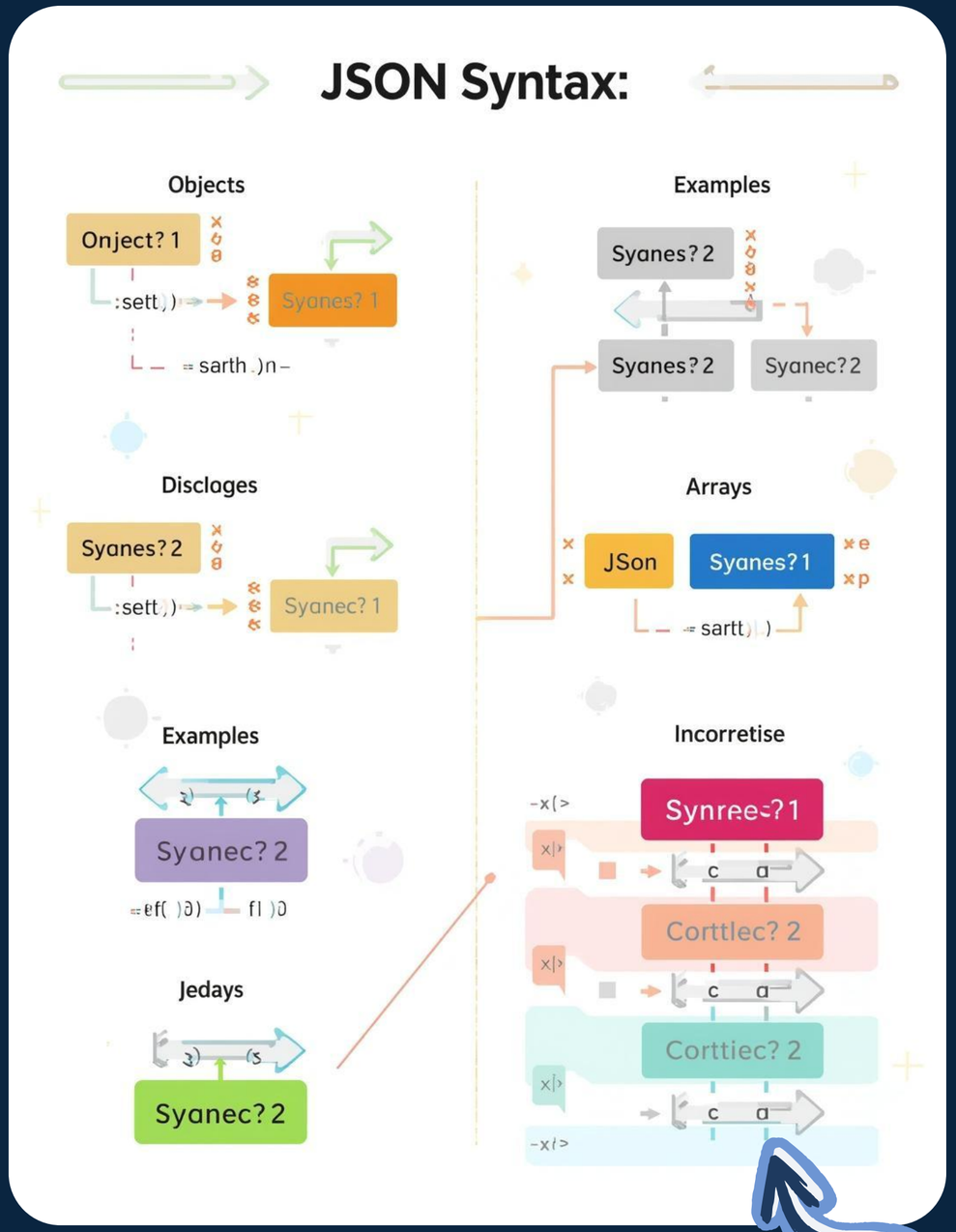
Esempio di JSON

Vediamo che JSON supporta nativamente array e oggetti, mentre XML richiede una struttura più complessa.

Oggetto Json: *collezione non ordinata di coppie chiave-valore.*

Array: *lista ordinata di valori.*

```json
{
  "utente": {
    "id": 123,
    "nome": "Mario Rossi",
    "email": "mario.rossi@example.com",
    "ordini": [
      {
        "id": 1001,
        "data": "2024-01-15",
        "totale": 89.90
      },
      {
        "id": 1002,
        "data": "2024-01-20",
        "totale": 129.50
      }
    ]
  }
}
```



Regole Sintattiche

- *Chiavi sempre tra doppi apici: "nome"* ✅ *non 'nome'* ❌
- *Niente virgola dopo ultimo elemento: [1, 2, 3]* ✅ *non [1, 2, 3,]* ❌
- *Solo doppi apici per stringhe: "testo"* ✅ *non 'testo'* ❌
- Nomenclatura case-sensitive

JSON è lo standard per le API REST per la sua semplicità e universalità.

Non è l'unico formato di rappresentazione delle risorse: si trovano comunemente anche XML o HTML.

Vantaggi e Svantaggi di JSON

Vantaggi di JSON

- Leggero (meno verboso di XML)
- Facile da leggere e scrivere
- Supporto nativo in JavaScript
- Parsing veloce
- Supportato da tutti i linguaggi moderni

Svantaggi di JSON

- No commenti
- No date native (si usano stringhe ISO 8601)
- No tipi complessi (tutto è stringa, numero o booleano)

Esercizio: oggetto libro

Costruiamo ora la rappresentazione JSON di un array di oggetti **"libro"** che abbiano le seguenti proprietà:



- **titolo**: *string*
- **genere**: *string*
- **autore**: *string*
- **editore**: *string*
- **numeroPagine**: *number*

PRO: organizzare i libri secondo il genere (suggerimento: ci sarà un oggetto contenitore per i generi, e ogni genere sarà un oggetto che conterrà un array di libri).

Usiamo un validatore JSON online per controllare il risultato: https://jsonlint.com/

Esercizio: libreria

Ora usiamo l’array appena creato all’interno di un oggetto **“libreria”** così composto:

- **nome**: *string*
- **indirizzo**: *string*
- **catalogoLibri**: *libro[ ]*

PRO: l’indirizzo diventa un oggetto con I campi via, citta, regione.

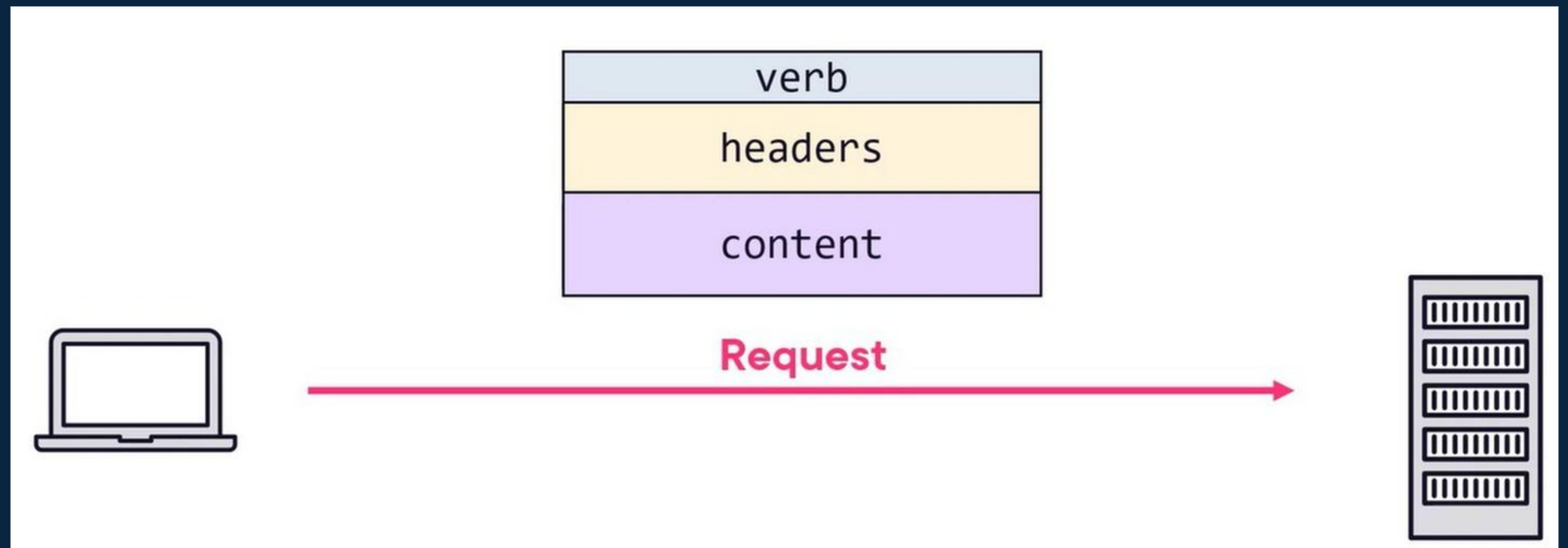
Controlliamo sempre il risultato sul validatore JSON https://jsonlint.com/



Panoramica Request e Response

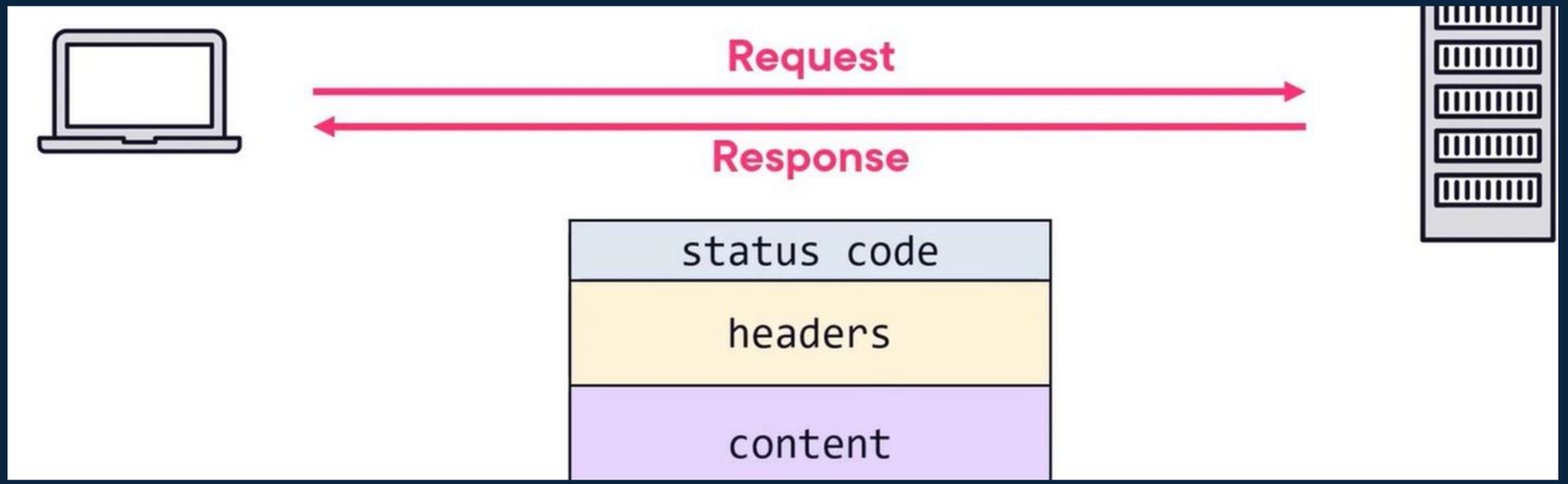
Vediamo dunque come si compongono i due oggetti cardine del protocollo HTTP e di tutta la comunicazione REST: **request** e **response**.

1) il client web (browser, p.es. Chrome) invia al server web (p.es. Apache) una richiesta (REQUEST), ad esempio inserendo nella barra del browser un indirizzo; parte una richiesta verso il server che contiene tre parti: verb/method (GET, POST, etc) , headers (content lenght: 10) , content : {"requestContent": “Ciao mondo”}



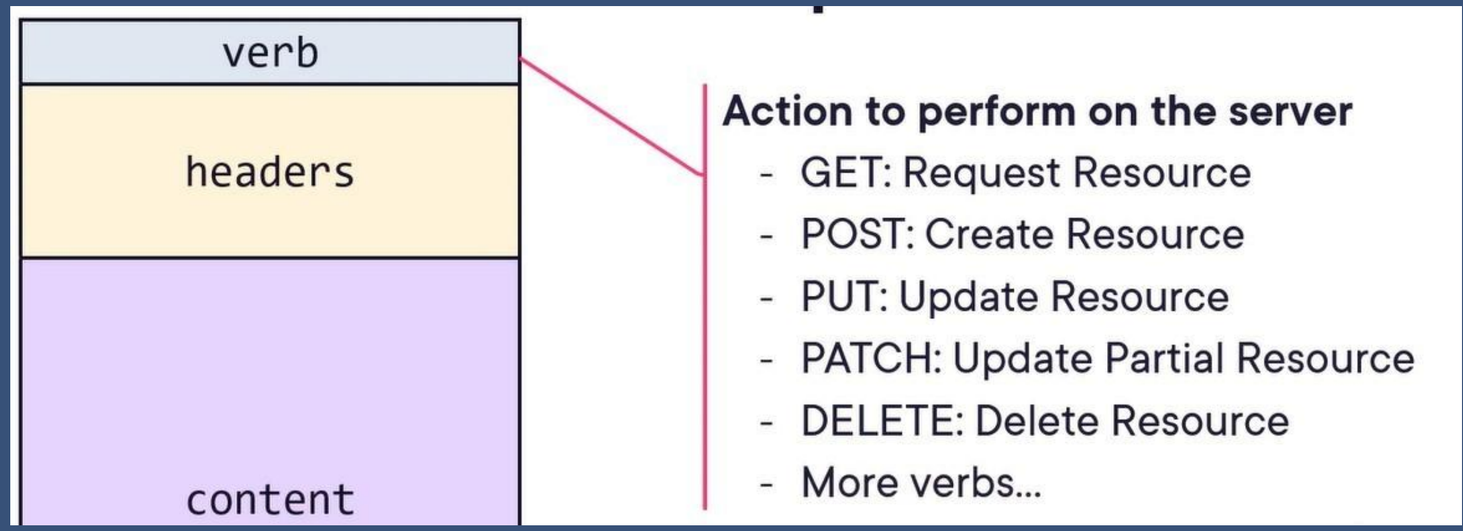
Panoramica Request e Response

2) il server web invia al client la risposta (RESPONSE) che contiene tre parti: status code (1xx, 2xx, 3xx,4xx,5xx) , headers (content type: text) , content : {"responseContent": “Ciao mondo”}



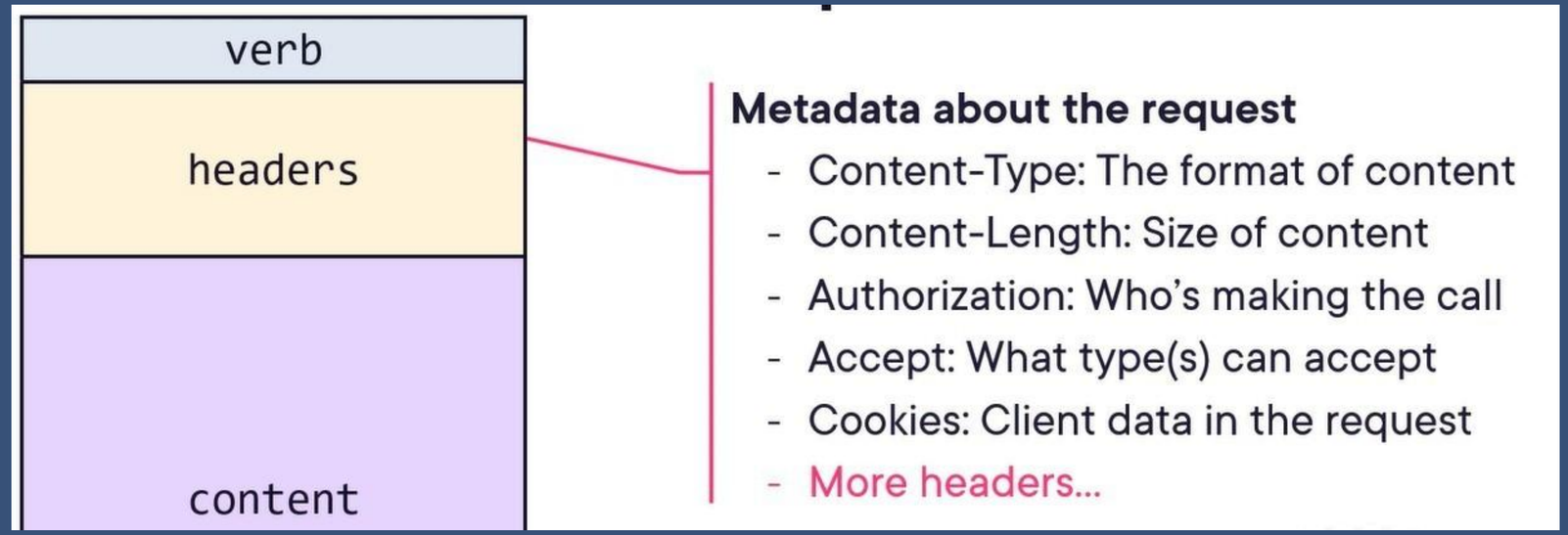
Analisi della Request (metodi)

VERB/METHOD : sono i metodi HTTP di cui abbiamo già parlato in precedenza



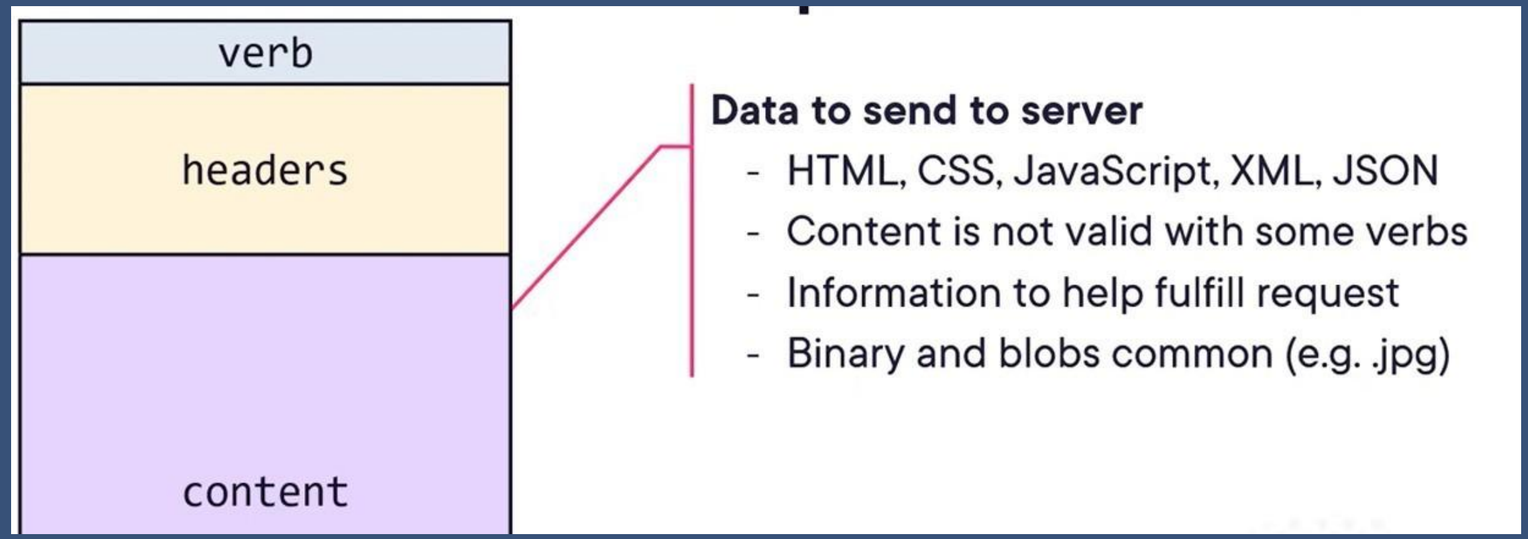
Analisi della Request (headers)

HEADERS : metadati che possono contenere informazioni sulla request (possiamo crearne anche di custom)



Analisi della Request (body)

CONTENT: quello che mandiamo al server; sono informazioni che possono servire al server (ad esempio dati presenti in un form , un oggetto JSON, un file da uploadare...)





Query Parameters

La chiamata GET ha una particolarità, ossia l'utilizzo dei query parameters: essi sono una parte dell'URL che permette di passare informazioni aggiuntive al server in una richiesta HTTP.

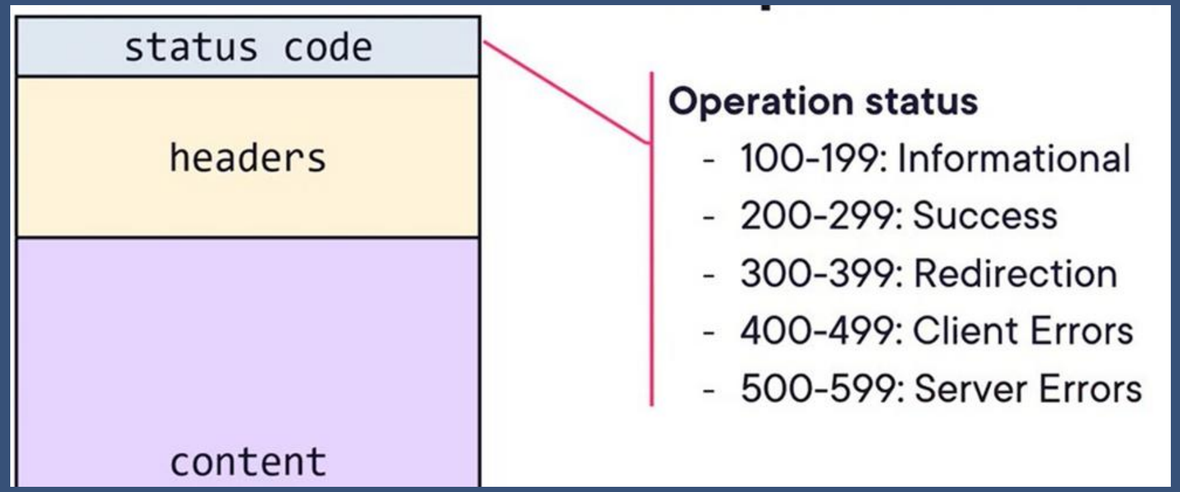
Si trovano dopo il simbolo ? nell'URL e sono formattati come coppie chiave=valore, separati da &:

http://api.example.com/books ? category=fiction & price_max=50 & sort=title

Analisi della Response: (codici di stato)

Passiamo ora alla Response.

STATUS CODE: un codice che indica lo stato della risposta (tutto ok, risorsa non trovata , errore). I codici sono predefiniti (*https://it.wikipedia.org/wiki/Codici_di_stato_HTTP*)



Analisi della Response: (codici di stato)

Codici Informativi: 1xx



Indicano che la richiesta è stata ricevuta e il server sta continuando a elaborarla.

- 100 Continue: La richiesta è in corso.
- 101 Switching Protocols: Il server sta cambiando protocollo come richiesto dal client.
- 102 Processing: La richiesta è stata ricevuta e il server sta elaborando, ma non ha ancora una risposta disponibile.

Analisi della Response: (codici di stato)

Successo: 2xx



Indicano che la richiesta è stata ricevuta, compresa e accettata.

- 200 OK: La richiesta è andata a buon fine.
- 201 Created: La richiesta ha portato alla creazione di una nuova risorsa.
- 204 No Content: La richiesta è stata elaborata, ma non ci sono contenuti da restituire.
- 206 Partial Content: Solo una parte della risorsa è stata restituita

Analisi della Response: (codici di stato)

Codici di Redirect: 3xx



Indicano che il client deve eseguire ulteriori azioni per completare la richiesta.

- 300 Multiple Choices: Ci sono più opzioni per la risorsa richiesta.
- 301 Moved Permanently: La risorsa è stata spostata permanentemente a un nuovo URI.
- 302 Found: La risorsa è stata trovata, ma è temporaneamente disponibile altrove.

Analisi della Response: (codici di stato)

Errore lato client: 4xx



Indicano che c'è stato un errore nella richiesta del client.

- 400 Bad Request: La richiesta non può essere elaborata a causa di sintassi errata.
- 401 Unauthorized: La richiesta richiede autenticazione.
- 403 Forbidden: Il server ha compreso la richiesta, ma rifiuta di autorizzarla.
- 404 Not Found: La risorsa richiesta non è stata trovata.

Analisi della Response: (codici di stato)



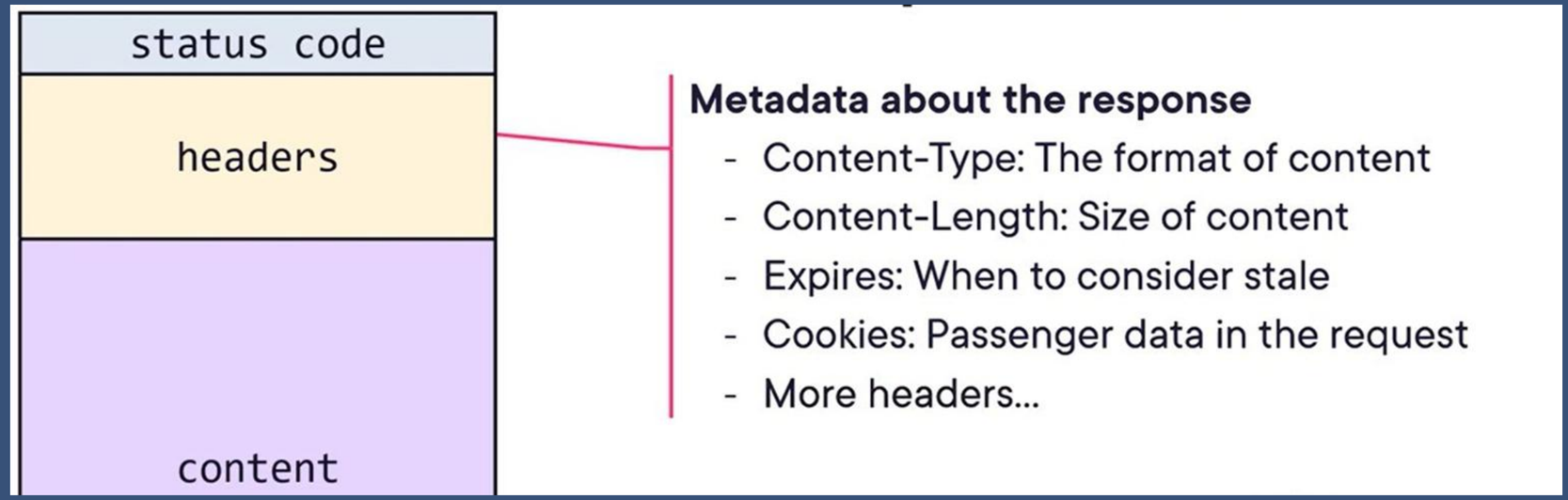
Errore lato server: 5xx

Indicano che il server ha incontrato un errore o è incapace di gestire la richiesta.

- 500 Internal Server Error: Errore generico del server.
- 502 Bad Gateway: Il server ha ricevuto una risposta non valida da un altro server.
- 503 Service Unavailable: Il server non è disponibile per il momento.
- 504 Gateway Timeout: Il server non ha ricevuto una risposta in tempo da un altro server..

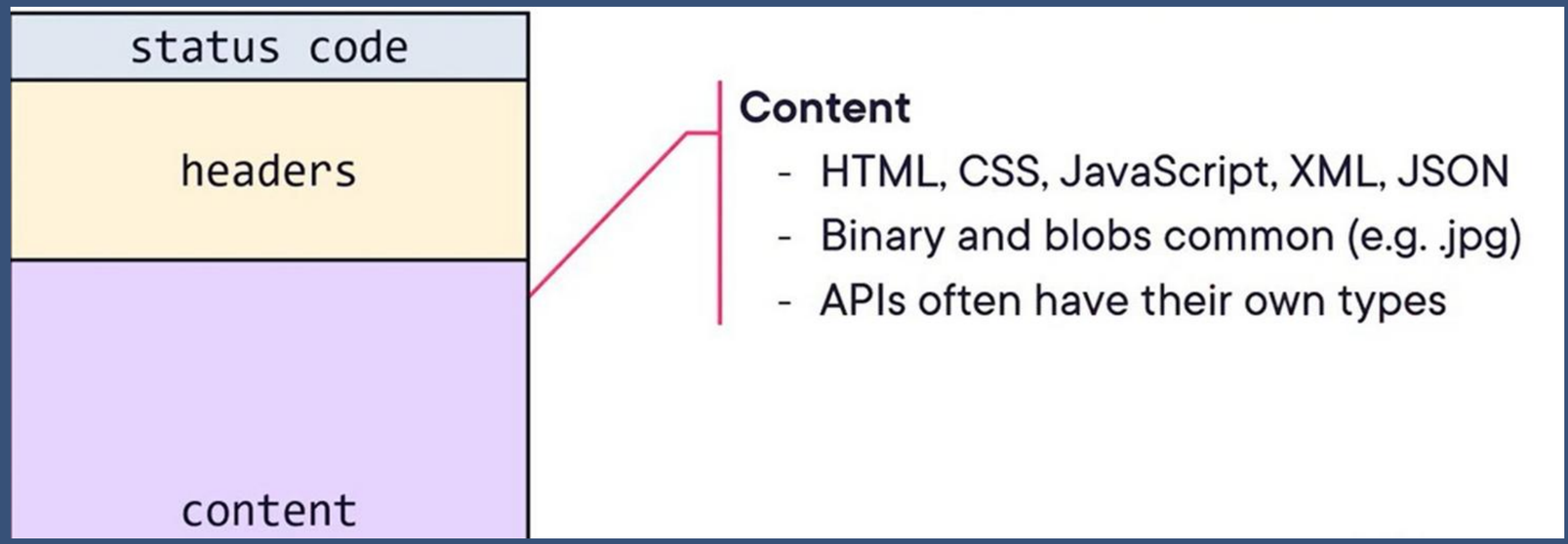
Analisi della Response: (headers)

HEADERS: metadati che possono contenere informazioni sulla response (possiamo crearne anche di custom)



Analisi della Response: (body)

CONTENT: il contenuto che il server restituisce al client (solitamente in formato JSON)

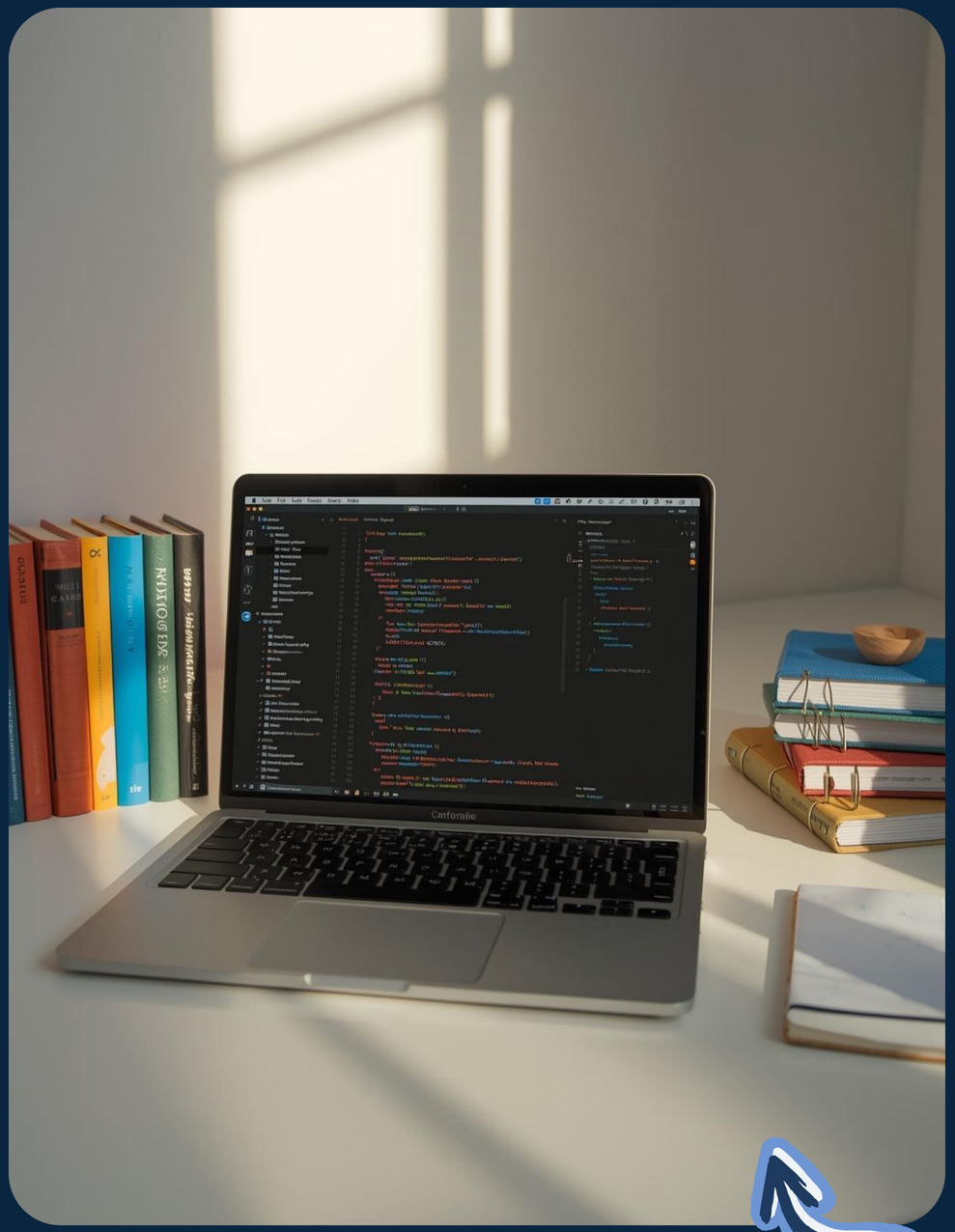


Cookies



C’è infine un altro tipo di dati che spesso viene creato dal server e memorizzato sul client tramite il browser: i COOKIES.
I cookies sono piccoli pezzi di dati che il server invia al client e che il browser memorizza e reinvia automaticamente nelle richieste successive. Vengono salvati fisicamente su file, e sono utilizzati principalmente per mantenere sessioni e preferenze utente (lingua, tema grafico, preferenze…).

Payload (o content, o body)	Headers	Cookies
Contenuto principale (JSON con dati)	Metadati, autenticazione, controllo cache	Sessioni, preferenze persistenti
Inviato ogni volta esplicitamente	Inviati ogni volta esplicitamente	Memorizzati dal browser e inviati automaticamente



Esercitazione finale

Ora usiamo gli oggetti che abbiamo creato per creare un semplice server node che esponga delle API (Application Programming Interface) per gestire la nostra libreria.

Ci serviranno:

- un IDE (Visual Studio Code)
- Node.js + Express



Risorse utili

TOOLS:

- Node.js (https://nodejs.org/docs)
- Express (https://expressjs.com)
- Postman (https://www.postman.com)

CORSI E TUTORIAL:

- MDN Web Docs (https://developer.mozilla.org/en-US/docs/Web/HTTP)
- RESTful API Design (https://restfulapi.net)



Grazie per l'attenzione!

*Docente: **Fabio Abbà***

fabio.abba@tc-web.it

Credits: Franco Grivet

franco.grivet@tc-web.it

Intervento realizzato da

ITS TECNOLOGIE DELL'INFORMAZIONE E DELLA COMUNICAZIONE

TC-Web Technology Consulting

Via Giovanni Schiaparelli 14,
10148 – Torino (TO)
E-mail: info@tc-web.it
Tel: 011 0466685